Methods: Computational Kernels, Model Scope, and Quantitative Provenance

Computational kernels as the sole authority for quantitative examples

Computational kernels as the sole authority for quantitative examples I

The `src/kernels` package implements the mathematics of Active Inference as pure, numerically stable, fully typed functions in natural-log (nats) units:

Computational kernels as the sole authority for quantitative examples I

- **Information** – surprise $-\ln p$, Shannon entropy $H(p) = -\sum_x p(x) \ln p(x)$, cross-entropy, Kullback-Leibler divergence $\mathrm{KL}(p\|q)$, and mutual information, following the original and textbook information-theory treatments (Shannon 1948; Cover and Thomas 2006).
- **Distributions** – normalization, a precision-parameterized softmax, and Dirichlet expectations; these are the small probability utilities needed by the active-inference and variational examples (Beal 2003).
- **Inference** – exact discrete Bayesian posteriors and the variational free-energy identity $F[q] = \mathbb{E}_{q(s)}[\ln q(s) - \ln p(o, s)] = D_{\mathrm{KL}}(q(s)\|p(s)) - \mathbb{E}_{q(s)}[\ln p(o \mid s)] = D_{\mathrm{KL}}(q(s)\|p(s \mid o)) - \ln p(o)$, together with the log-evidence and the evidence lower bound (Bishop 2006; Blei et al. 2017; Buckley et al. 2017).
- **Active inference** – expected free energy $G(\pi)$ with the convention that lower G is preferred, policy selection $q(\pi) = \mathrm{softmax}(-\gamma G(\pi))$ where γ is policy precision, and a `DiscreteActiveInferenceAgent` running a deterministic perception -> policy -> action loop (Friston et al. 2015; K. J. Friston et al. 2017; Da Costa et al. 2020; Sajid et al. 2021; Millidge et al. 2021; Champion et al. 2026).

Scope of the implemented kernels I

The implemented kernels intentionally stop at a small, discrete, one-step reference agent. Generalized free energy and action-oriented model learning are important adjacent targets, but they remain scope markers for future kernels rather than hidden features of this release (Parr and Friston 2019; Tschantz et al. 2020).

Evidence boundary and analytic checks I

The resulting evidence is model-specific twice over. First, a tested function establishes that the implementation matches its stated mathematical identity under the declared assumptions and conventions. Second, a fixed-input simulation shows how that selected model behaves under its priors, likelihoods, preferences, precision, and policy set. It does not show that the same assumptions describe a biological brain, a human learner, or all formulations of Active Inference. This distinction follows the separation between process-theory construction, implementation choice, and empirical-status assessment in the Active-Inference literature (K. Friston et al. 2017; Da Costa et al. 2020; Aguilera et al. 2022; Hodson et al. 2024).

Evidence boundary and analytic checks I

Kernel correctness is asserted against analytic identities: the entropy of a uniform distribution over n outcomes equals $\ln n$; KL is non-negative and zero only at equality; the free-energy bound is tight at the true posterior; and the expected-free- energy preference is regime-dependent rather than unconditional. The reproducibility section records the exact coverage and mock-free gates that must run before treating this build as release evidence.

Statistical boundary: deterministic validation without learner-outcome inference

Statistical boundary: deterministic validation without learner-outcome inference I

Active Skillference does not report inferential statistics, learner outcomes, effect sizes, p-values, confidence intervals, or population-level empirical claims. Its evidence has three narrower classes. First, deterministic enumeration validates graph shape, source-matrix coverage, figure/artifact wiring, web math, roadmap state and SkillTree export counts. Second, computational kernels and simulation sweeps show model-specific mathematical behavior under fixed inputs, such as noisy-sensor posterior sensitivity and the sample/commit expected-free-energy boundary. Third, browser smoke checks and optional online source-health checks are delivery and currentness evidence. Those checks can reveal stale routes, missing artifacts, failed MathJax rendering or unreachable source locators; they are not evidence that learners improved or that a biological process theory is empirically confirmed.

Statistical boundary: deterministic validation without learner-outcome inference I

The next two layers apply that evidence discipline to authored content. The first keeps numbers traceable to kernels; the second keeps citations traceable to explicit support roles.

Provenance binding from kernel registry to learner-facing prose

Provenance binding from kernel registry to learner-facing prose I

Worked examples in `src/kernels/examples.py` run the kernels on fixed, documented inputs and register each result in a `ClaimRegistry` under a stable name. Skill descriptions, assessment stems, correct numeric answer options, and manuscript prose reference these values only through `claim:NAME` tokens, following the reproducible-reporting pattern of binding prose to executable analysis rather than hand-maintained results (Xie 2015; Sandve et al. 2013). Three gates enforce integrity:

Provenance binding from kernel registry to learner-facing prose I

1. **Resolution** – every token must resolve to a registered claim, or rendering raises.
2. **Fail-closed export** – the SkillTree exporter calls the resolver; an unbacked token aborts the export with nothing written.
3. **Bare-number linter** – any decimal result with two or more fractional digits sitting in plain prose (not inside a claim token or `$...$` math) is reported as a violation. Correct answer options are scanned as well, so a derived value cannot bypass provenance by hiding in an answer key; wrong distractors remain free-form.

Provenance binding from kernel registry to learner-facing prose I

We use "provenance" in a PROV-inspired sense: the system records entities, activities, and agents sufficient for local traceability, but it does not claim conformance to a full W3C PROV interchange implementation. In Active Skillference terms, the entities are claim records, rendered skills, manuscript fragments, figures, and SkillTree exports; the activities are kernel execution, rendering, export, and validation; and the agents are the source files and scripts that produce each artifact. The PROV overview, data model, and constraints recommendation supply the formal vocabulary for derivation, attribution, bundles, and validity checks, while this project implements a project-local claim registry and release ledger rather than a general PROV service (Groth and Moreau 2013; Moreau and Missier 2013; Cheney et al. 2013). Workflow Run RO-Crate and the related Provenance Run Crate profile provide a useful interoperability target for the same boundary:

they package workflow-run metadata, inputs, outputs, parameters, and execution provenance as RO-Crate records. Active Skillference's `claims.json`, manuscript variables, figure registry, and release ledger play a narrower analogous role inside this repository. The project does not emit a conforming RO-Crate, validate against those profiles, or expose a general provenance interchange endpoint; the comparison is a roadmap-level mapping from the local artifact chain to a recognized workflow-provenance model (Leo et al. 2024).

Provenance binding from kernel registry to learner-facing prose I

This mechanism sits in the lineage of literate and reproducible computational reporting: Knuth made code and explanation a joint design object, Sweave/knitr-style workflows bind prose to executable analysis, and reproducible-research arguments treat rerunnable computational claims as a minimum standard when independent replication is difficult (Knuth 1984; Xie 2015; Peng 2011; Sandve et al. 2013). Active Skillference moves that binding to a registry and build gate that protects both the manuscript and the exported curriculum.

Provenance binding from kernel registry to learner-facing prose I

Provenance binding and kernel testing are complementary halves of the same pedagogical contract. Binding establishes that a value a learner sees is a kernel's output rather than a hand-typed figure; the deterministic kernel tests establish that the kernel computes the intended quantity in the first place. A learner-facing number is therefore only as sound as the kernel behind it, which is why the integrity story rests on the tested kernels and not on the binding alone: the binding makes a number traceable, and the tests are what make that traceability worth anything. The export and render gates abort when a kernel token is missing or a result was typed into prose, so an unbacked number cannot reach a learner unnoticed.

Provenance binding from kernel registry to learner-facing prose I

Numeric traceability is not enough for a scholarly curriculum. The prose also needs to show which cited sources support lineage, method, delivery, critique, and scope claims.